

NoSQL Programming

Trainer : Pradnyaa S Dindorkar

Email: pradnya@sunbeaminfo.com



- Introduction to NoSQL
- MongoDB
- Redis
- Cassandra



What are we going to cover?

- What is NoSQL and how is it different from RDBMS?
- What is JSON?
- Introduction to MongoDB
- JSON vs BSON
- MongoDB Fundamentals
- Basic CRUD operations
- Performance optimization
- Data Modeling
- Aggregation Pipeline
- Introduction to Geospatial Queries



- What is database?
- Quick recap of sql,
- limitation of sql,
- intro of nosql ,
- CAP theorem and BASE theorem
- types of nosql
- Advantages of NoSQL
- Disadvantages of NoSQL
- NoSQL Scenarios,
- SQL vs NoSQL



What is Database ?

- A database is an organized collection of data, generally stored and accessed electronically from a computer system
- A database refers to a set of related data and the way it is organized
- Database Management System
 - Software that allows users to interact with one or more databases and provides access to all of the data contained in the database
- Types
 - RDBMS (SQL)
 - NoSQL



- The idea of RDBMS was borne in 1970 by E. F. Codd
- The language used to query RDBMS systems is SQL (Sequel Query Language)
- RDBMS systems are well suited for structured data held in columns and rows, which can be queried using SQL
- Supports: DML, DQL, DDL, DTL, DCL
- The RDBMS systems are based on the concept of ACID transactions
 - **Atomic**: implies either all changes of a transaction are applied completely or not applied at all
 - **Consistent**: data is in a consistent state after the transaction is applied
 - **Isolated**: transactions that are applied to the same set of data are independent of each other
 - **Durable**: the changes are permanent in the system and will not be lost in case of any failures





- Scalability is the ability of a system to expand to meet your business needs.
- Scalability describes a elasticity of the system, ability to adapt to change and demand.
- Good scalability ensures the quality of your service.

Vertical Scaling



CPU: 3 , RAM: 6G



CPU: 2 , RAM: 4G



CPU: 1 , RAM: 2G



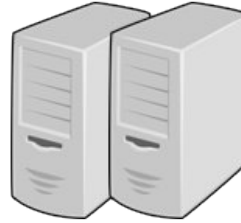
Vertical scaling describes adding more resources to your current machines.

- ✓ increase memory in the system
- ✓ expanding storage by adding hard drives
- ✓ upgrading the CPUs.
- ✓ upgrading network speed.

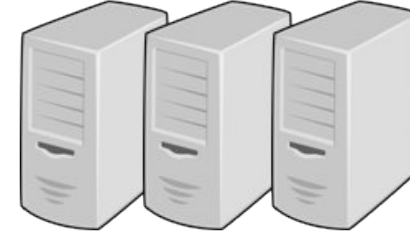
Horizontal Scaling



1 PC (CPU: 1, RAM:2G)



2 PC (CPU: 1, RAM:2G)



3 PC (CPU: 1, RAM:2G)



Horizontal Scaling refers to adding additional nodes or machines to your infrastructure to cope with new demands.

- ✓ adding a new computer to a distributed software application

Structured Data



Structured data is highly specific and is stored in a predefined format, well-defined data model.

Example: Employee table in the database ,
Banking transaction data .

Table: Transaction				+: Credit, -: Debit	
Transaction ID	Transaction Date Time	User ID	Account ID	Amount	Account Balance
1000001	01/04/2012 09:10:19	2	1	3100.00	4,300.21
1000002	01/04/2012 11:10:19	4	3	5800.00	6,412.44
1000003	01/04/2012 12:10:19	3	4	1200.00	307.85
1000004	01/04/2012 13:10:19	1	5	2500.00	229.87
1000005	02/04/2012 09:10:19	5	1	-50.00	4,250.21
1000006	02/04/2012 11:10:19	3	3	-100.00	612.44
1000007	02/04/2012 14:10:19	1	6	810.00	-99,119.91
1000008	03/04/2012 09:10:19	3	1	-50.00	4,200.21
1000009	03/04/2012 11:10:19	1	3	-100.00	512.44
1000010	03/04/2012 14:10:19	5	6	810.00	-98,309.91

Unstructured Data



Unstructured data is information that is not arranged according to a preset data model or schema. Unstructured data is very difficult to analyze. It required AI and machine learning , new software tools

Example: Videos, Audio files , Word, PDF, Text, Media logs.



Semi-structured Data



Semi-structured data contains some level of organization or structure, but does not conform to a rigid schema or data model

Example: XML data, JSON data

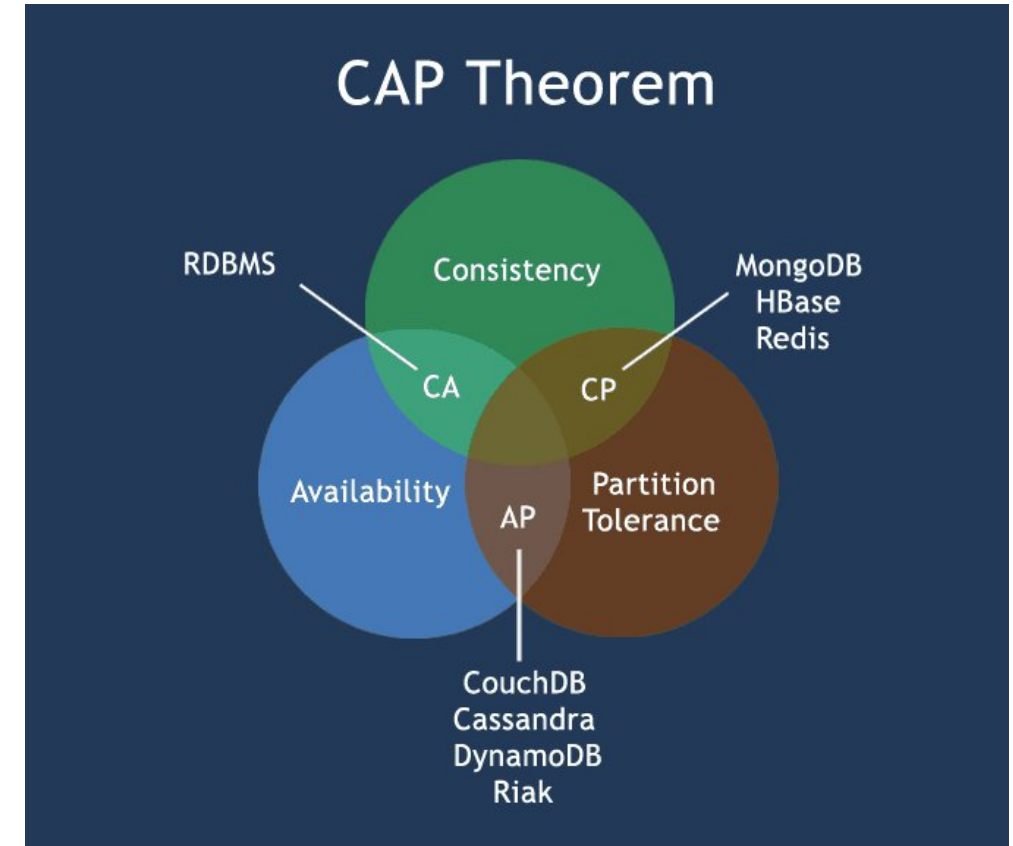
```
{
  "empoyee":
  [
    {
      "name": "sujay",
      "age": 56,
      "married": true,
      "contact": 9887667767 ,
      "Language": ["C++", "JAVA", "Python", "Kotlin", "Go"]
    },
    {
      "name": "ajay",
      "age": 28,
      "married": false,
      "Favorite-Sports": ["Football", "Basketball"]
    }
  ]
}
```

- Stands for Not Only SQL
- NoSQL is a term used to refer to non-relational databases
- The term NoSQL was coined by Carlo Strozzi in 1998 .
- Does not use any declarative query language
- No predefined schema → no restriction over the schema
- Unstructured and unpredictable data
- Supports eventual consistency rather than ACID properties
- Prioritizes high performance, high availability and scalability
- In contrary to the ACID approach of traditional RDBMS systems, NoSQL solves the problem using an approach popularly called as BASE



CAP Theorem

- Also known as Brewer's theorem states that it is impossible for a distributed data store to simultaneously provide more than two out of the following three guarantees
 - Consistency
 - Data is consistent after operation
 - After an update operation, all clients see the same data
 - Availability
 - System is always on (i.e. service guarantee), no downtime
 - Partition Tolerance
 - System will continue to function even if it is partitioned into groups of servers that are not able to communicate with one another



BASE Theorm

- Eric Brewer coined the BASE acronym
- BASE means
 - **Basically Available**: means the system will be available in terms of the CAP theorem.
 - **Soft state** indicates that even if no input is provided to the system, the state will change over time. This is in accordance to eventual consistency.
 - **Eventual consistency**: means the system will attain consistency in the long run, provided no input is sent to the system during that time.



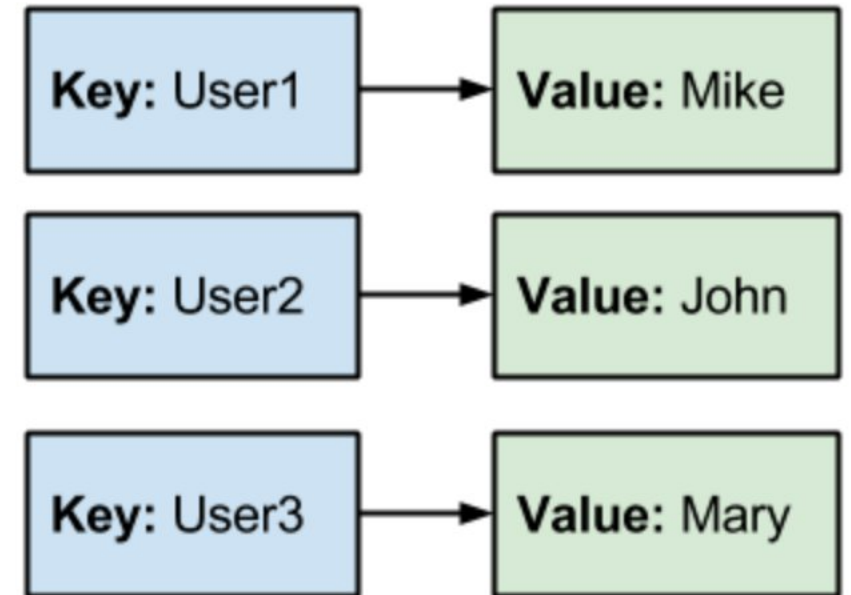
NoSQL Types

- Following are the types of NoSQL database based on how it stores the data
 - Key Value
 - Document DB
 - Column Based DB
 - Graph DB



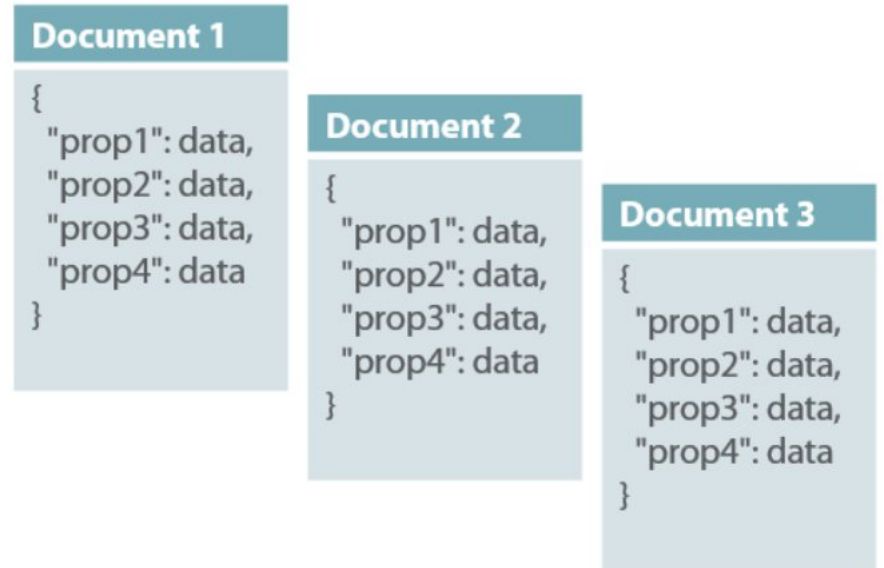
Key Value Database

- Data is stored as keys and values
- Provides super fast queries and ability to scale almost infinite data and performance level
- No relationships and weak/no schema
- E.g.
 - DynamoDB
 - Redis
 - Couchbase



Document DB

- Data is stored as Documents
- Document: collection of key-value pairs with structure
- Great performance when interacting with documents
- E.g.
 - MongoDB
 - CouchDB
 - RethinkDB



Column Based DB

- Data is stored as columns rather than rows
- Fast queries for datasets
- Slower when looking at individual records
- Great for warehousing and analytics
- E.g.
 - Redshift
 - Cassandra
 - HBase
 - Vertica

ID	Name
0001	Roffle
0002	Penny
0003	Winkie



Graph DB

- Designed for dynamic relationship
- Stores data as nodes and relationships between those nodes
- Ideal for human related data like social media
- Often as custom query language
- E.g.
 - Neo4J
 - Giraph
 - OrientDB



Advantages of NoSQL

- High scalability
 - This scaling up approach fails when the transaction rates and fast response requirements increase. In contrast to this, the new generation of NoSQL databases is designed to scale out (i.e. to expand horizontally using low-end commodity servers).
- Manageability and administration
 - NoSQL databases are designed to mostly work with automated repairs, distributed data, and simpler data models, leading to low manageability and administration.
- Low cost
 - NoSQL databases are typically designed to work with a cluster of cheap commodity servers, enabling the users to store and process more data at a low cost.
- Flexible data models
 - NoSQL databases have a very flexible data model, enabling them to work with any type of data; they don't comply with the rigid RDBMS data models. As a result, any application changes that involve updating the database schema can be easily implemented.



Disadvantages of NoSQL

- **Maturity**
 - Most NoSQL databases are pre-production versions with key features that are still to be implemented. Thus, when deciding on a NoSQL database, you should analyze the product properly to ensure the features are fully implemented and not still on the To-do list.
- **Support**
 - Support is one limitation that you need to consider. Most NoSQL databases are from start-ups which were open sourced. As a result, support is very minimal as compared to the enterprise software companies and may not have global reach or support resources.
- **Limited Query Capabilities**
 - Since NoSQL databases are generally developed to meet the scaling requirement of the web-scale applications, they provide limited querying capabilities. A simple querying requirement may involve significant programming expertise.
- **Administration**
 - Although NoSQL is designed to provide a no-admin solution, it still requires skill and effort for installing and maintaining the solution.
- **Expertise**
 - Since NoSQL is an evolving area, expertise on the technology is limited in the developer and administrator community.



SQL vs NoSQL

	SQL	NoSQL
Types	All types support SQL standard	Multiple types exists, such as document stores, key value stores, column databases, etc
History	Developed in 1970	Developed in 2000s
Examples	SQL Server, Oracle, MySQL	MongoDB, HBase, Cassandra
Data Storage Model	Data is stored in rows and columns in a table, where each column is of a specific type	The data model depends on the database type. It could be Key-value pairs, documents etc
Schemas	Fixed structure and schema	Dynamic schema. Structures can be accommodated
Scalability	Scale up approach is used	Scale out approach is used
Transactions	Supports ACID and transactions	Supports partitioning and availability
Consistency	Strong consistency	Dependent on the product [Eventual Consistency]
Support	High level of enterprise support	Open source model
Maturity	Have been around for a long time	Some of them are mature; others are evolving



NoSQL Scenarios

- When to use NoSQL ?
 - Large amount of data (TBs)
 - Many Read/Write operations
 - Economical scaling
 - Flexible Schema
- Examples
 - Social Media
 - Recordings
 - Geospatial analysis
 - Information processing
- When NOT to use NoSQL ?
 - Need ACID transactions
 - Fixed multiple relations
 - Need joins
 - Need high consistency
- Examples
 - Financial transactions
 - Business operations





MongoDB Overview

- Developed by 10gen in 2007
- Publicly available in 2009
- Open-source database which is controlled by 10gen
- Document oriented database → stores JSON documents
- Stores data in binary JSON



Install MongoDB

- Install MongoDB by downloading community edition
 - (<https://www.mongodb.com/download-center/community>)
- Linux and Mac Users
 - Extract the downloaded file somewhere in the disk.
 - Set the environment path to use the tools without going to the bin directory in the ~/.bash_profile or ~/.bashrc file.
- Ubuntu (20.04) Mongo installation
 - terminal> `wget -qO - https://www.mongodb.org/static/pgp/server-4.4.asc | sudo apt-key add -`
 - terminal> `echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/4.4 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-4.4.list`
 - terminal> `sudo apt-get update`
 - terminal> `sudo apt-get install -y mongodb-org`
- Windows Users
 - Install the MongoDB by following all the steps in the installation wizard
 - Set the the environment path to include the <path>/bin



JSON

- Java Script Object Notation
- Hierarchical way of organizing data
- Defined as part of the JS language by JavaScript creator Douglas Crockford (2000).
- JavaScript objects are associative containers, wherein a string key is mapped to a value
- JSON shows up in many different cases.
 - APIs
 - Configuration files
 - Log messages
 - Database storage
- JSON is not ideal for usage inside of a database.
 - JSON is a text-based format, and text parsing is very slow
 - JSON's readable format is far from space-efficient, another database concern
 - JSON only supports a limited number of basic data types
- Mongo stores JSON data into Binary form.



JSON have

- String -> "hello" "name"
- Numbers -> 30 1.5 -40 1.2e10
- Booleans -> true false
- Array -> ["Jan", "feb", "Mar"] [10 ,20 ,30]
- Object -> {"key","value"} {"age": 20}

Key is always a string

At top level typically have an array or object



BSON

- BSON simply stands for “Binary JSON”
- Binary structure encodes type and length information, which allows it to be parsed much more quickly
- It has been extended to add some optional non-JSON-native data types
- It allows for comparisons and calculations to happen directly on data
- MongoDB stores data in BSON format both internally, and over the network
- Anything you can represent in JSON can be natively stored in MongoDB

	JSON	BSON
Encoding	UTF-8 String	Binary
Data Support	• String • Boolean • Number • Array	• String • Boolean • Number • Integer • Float • Long • Decimal • Array • Date • Raw Binary
	Human and Machine	Machine Only



```
{  
  "_id": 1,  
  "name" : { "first" : "Sanjay", "last" : "Pawar" },  
  "Language" : [ "C++", "JAVA", "Python", "Kotlin", "Go" ],  
  "awards" : [  
    { "Winner" : "Best developer", "year" : 1998 },  
    { "2nd Runner-Up" : "Best Programmer", "year" : 2000 }  
  ]  
}
```



Mongo Server and Client

- MongoDB server (mongod) is developed in C, C++ and JS.
- MongoDB data is accessed via multiple client tools
 - mongo : client shell (JS).
 - mongofiles : stores larger files in GridFS.
 - mongoimport / mongoexport : tools for data import / export.
 - mongodump / mongorestore : tools for backup / restore.
- MongoDB data can be accessed in application through client drivers available for all major programming languages e.g. Java, Python, Ruby, PHP, Perl, ...
- Mongo shell is follows JS syntax and allow to execute JS scripts.



MongoDb: Data Types

data	bson	values
null	10	
boolean	8	true, false
number	1 / 16 / 18	123, 456.78, NumberInt("24"), NumberLong("28")
string	2	"...."
date	9	new Date(), ISODate("yyyy-mm-ddThh:mm:ss")
array	4	[..., ..., ..., ...]
object	3	{ ... }



MongoDB Terminology

- **Database**

- This is a container for collections like in RDMS wherein it is a container for tables
- Each database gets its own set of files on the file system
- A MongoDB server can store multiple databases

- **Collection**

- This is a grouping of MongoDB documents
- A collection is the equivalent of a table which is created in any other RDMS such as Oracle or MS SQL
- Collections don't enforce any sort of structure

- **Document**

- A record in a MongoDB collection is basically called a document
- The document, in turn, will consist of field name and values

- **Field**

- Field names are strings
- A name-value pair in a document
- A document has zero or more fields
- Fields are analogous to columns in relational databases



Document

- MongoDB stores data records as BSON documents
- Maximum size of document is 16MB
- Restrictions
 - The field name `_id` is reserved for use as a primary key
 - Field names **cannot** contain the null character
 - Top-level field names **cannot** start with the dollar sign (\$) character



_id field

- Each document requires a unique _id field that acts as a primary key
- If an inserted document omits the _id field, the MongoDB driver automatically generates an ObjectId for the _id field
- Behaviors
 - By default, MongoDB creates a unique index on the _id field during the creation of a collection
 - The _id field is always the first field in the documents. If the server receives a document that does not have the _id field first, then the server will move the field to the beginning.
 - The _id field may contain values of any BSON data type, other than an array
- Autogenerated _id (of type ObjectId) will be of 12 bytes which contains
 - Timestamp: 4 bytes
 - Machine Id: 3 bytes
 - Process Id: 2 bytes
 - Counter: 3 bytes



CRUD operations



Database Operations

- List existing databases
 - > **show dbs**
 - > **show databases**
- Create and use database
 - > **use <db name>**
- Get the selected database name
 - > **db**
- Show the database statistics
 - > **db.stats()**



Collection operations

- Get the list of collections
> **show collections**
- Create Collection
> **db.createCollection('contacts')**
- Drop Collection
> **db.contacts.drop()**



Create Document (Insert data)

- Create one document

```
> db.contacts.insert({ name: 'ravi', mobile: '7709859986' })
```

- Create many documents

```
> db.contacts.insertMany([  
  { name: 'contact 1', address: 'pune' },  
  { name: 'contact 2', address: 'mumbai' }  
])
```

- Note: if you are passing the `_id` field, make sure that it is unique. If it is not unique, the document will not get inserted



Read/Find Documents (Query data)

- Find documents
> **db.contacts.find()**
- Returns cursor on which following operations allowed
 - **hasNext()**: returns if cursor can iterate further
 - **next()**: returns the next document
 - **skip(n)**: skips first n documents
 - **limit(n)**: limit the result to n
 - **count()**: returns the count of result
 - **toArray()**: returns an array of document
 - **forEach(fn)**: Iterates the cursor to apply a JavaScript function to each document from the cursor
 - **pretty()**: Configures the cursor to display results in an easy-to-read format
 - **sort()**: sorts documents
- Shell by default returns 20 records. Press "it" for more results





Data Modeling



- Embedded Data Models

- { name : { fname : "Nisha", lname : "Shinde" }, age : 44 }
- Suitable for one-to-one or one-to-many relationship.
- Faster read operation. Related data fetch in single db operation.
- Atomic update of document.
- Document growth reduce write performance and may lead to fragmentation.



Data Modeling

- Normalized Data Models

- contacts → { _id: 11, email : "Jay@gmail.com", mobile: "9527331338" }
- persons → { _id: 1, name : "Jay Patil", contact: 11 }
- Preferred for complex many-to-many relationship.
- Reduce data duplication.
- Can use DBRef() to store document reference:

contacts → { _id: 11, email : "Jay@gmail.com", mobile: "9527331338" }

■ persons → { _id: 1, name: "Jay Patil", contact: { \$db : "test", \$ref : "contacts", \$id: 11 } }

- DBRef() are not supported in all client-drivers.



Aggregation Pipeline



Overview

- Aggregation operations process data records and return computed results
- Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result
- Documents enter a multi-stage pipeline that transforms the documents into aggregated results
- Pipeline
 - The MongoDB aggregation pipeline consists of stages
 - Each stage transforms the documents as they pass through the pipeline
 - Pipeline stages do not need to produce one output document for every input document
 - e.g., some stages may generate new documents or filter out documents



Operators

- \$project: select columns (existing or computed)
- \$match: where clause (criteria)
- \$group: group by
 - { \$group: { _id: <expr>, <field1>: { <accum1> : <expr1> }, ... } }
 - The possible accumulators are: \$sum, \$avg, etc.
- \$unwind: extract array elements from array field
- \$lookup: left outer join
- \$out: put result of pipeline in another collection (last operation)





Today's topics

- Query data
- Filtering results
- Query Projection
- Update Document



Read/Find Documents (Query data)

- Find documents
 - > **db.contacts.find()**
- Returns cursor on which following operations allowed
 - **hasNext()**: returns if cursor can iterate further
 - **next()**: returns the next document
 - **skip(n)**: skips first n documents
 - **limit(n)**: limit the result to n
 - **count()**: returns the count of result
 - **toArray()**: returns an array of document
 - **forEach(fn)**: Iterates the cursor to apply a JavaScript function to each document from the cursor
 - **pretty()**: Configures the cursor to display results in an easy-to-read format
 - **sort()**: sorts documents
- Shell by default returns 20 records. Press "it" for more results



Filtering results

> **db.contacts.find({ name: 'ravi' })** : select * from contacts where name = 'ravi';
> **db.contacts.find({ name: /ravi/ })** : select * from contacts where name like '%ravi%';

- Relational operators
 - \$eq, \$ne, \$gt, \$lt, \$gte, \$lte, \$in, \$nin
- Logical operators
 - \$and, \$or, \$nor, \$not
- Element operators
 - \$exists, \$type
- Evaluation operators
 - \$regex, \$where, \$mod
- Array operators
 - \$size, \$elemMatch, \$all, \$slice



Query Projection

- Projection: selecting required fields while finding the documents
- E.g.
 - `db.contacts.find({}, {name: 1})`
- Required fields can be added the projection list with value 1
- Non-required fields can be added the projection list with value 0
- Can not mix both required and non-required in the projection



Update Document

- Syntax
 - `db.<collection>.update(criteria, newObject)`
- E.g.
 - `> db.contacts.update({ name: 'ravi' }, { $set: { address: 'Pune' } })`
- Update operators
 - `$set`, `$inc`, `$push`, `$each`, `$pull`
- In place updates are faster (e.g. `$inc`) than setting new object





Mongo – Indexes

- GeoSpatial Queries
- Capped Collections

Validators

Mongo – GridFS

Mongo Python Connectivity

Optimization



Mongo - Indexes

- `db.books.find( { "subject" : "C" } ).explain(true);`
- `explain()` → explains the query execution plan.
- Above query by default does full collection scan, hence slower.
- `db.books.createIndex( { "subject" : 1 } );`
- Searching on indexed columns reduces query execution time.
- Options can be provided (2nd arg): `{ unique : true }`
 - Duplicate values are not allowed in that field.
- By default `"_id"` field is indexed in mongodb (unique index).
- `db.books.getIndexes();`
- `db.books.dropIndex({ "subject" : 1 });`



Indexes - Types

- Regular index (Single Key)
- Composite index
- Unique index
- TTL index
- Geospatial indexes



Mongo - GeoSpatial Queries

- mongodb support three types of geo-spatial queries:
 - 2-d index: traditional long-lat. used in older mongodb (2.2-).
 - 2-d sphere index: data can be stored as GeoJSON.
 - geo-haystack: query on very small area. not much used.

GeoJSON stores geometry and coordinates: <https://geojson.io>

- { type: "geometry", coordinates: [long, lat] };
 - { type: "Point", coordinates: [73.86704859999998, 18.4898445] };
- Possible geometry types are:
 - Point, LineString, MultiLineString, Polygon
- allowed queries:
 - inclusion - \$geoWithin
 - intersection - \$geoIntersects
 - proximity - \$geoNear



Mongo - GeoSpatial Queries

- For faster execution create geo-index :
 - `db.busstops.createIndex( { location : "2dsphere" } );`
- Example proximity query:

```
db.busstops.find( { location : { $near : {  
    $geometry : { type : "Point" ,  
        coordinates : [73.86704859999998, 18.4898445]  
    },  
    $maxDistance : 200  
} } } );
```



Mongo - Capped Collections

- Capped collections are fixed sized collections for high-throughput insert and retrieve operations.
- They maintain the order of insertion without any indexing overhead.
- The oldest documents are auto-removed to make a room for new records. The size of collection should be specified while creation.
- The update operations should be done with index for better performance. If update operation change size, then operation fails.
- Cannot delete records from capped collections. Can drop collection.
- Capped collections cannot be sharded.
- `db.createCollection("logs", { capped: true, size: 4096 });` → if size is below 4096, 4096 is considered. Higher sizes are roundup by 256.



Mongo - GridFS

- GridFS is a specification for storing/retrieving files exceeding 16 MB.
- GridFS stores a file by dividing into chunks of 255 kb. When queried back, driver collect the chunks as requested. Query can be range query. Due to chunks file can be accessed without loading whole file in memory.
- It uses two collections for storing files i.e. fs.chunks, fs.files.
- It is also useful to keep files and metadata synced and deployed automatically across geographically distributed replica set.
- GridFS should not be used when there is need to update contents of entire file atomically.
- It can be accessed using **mongofiles** tool or compliant client driver.



Mongo - GridFS

- **fs.chunks**
 - `_id, files_id, n, data`
- **fs.files**
 - `_id, length, chunkSize, updateDate, md5, filename, contentType`
- **Files can be searched using**
 - `db.fs.files.find( { filename: myFileName } );`
 - `db.fs.chunks.find( { files_id: myFileID } ).sort( { n: 1 } )`
 - GridFS automatically create indexes for faster search.
- **mongofiles:**
 - `Mongofiles -d test put test.jpg`
 - `Mongofiles -d test get test.jpg`





Introduction

- Google BigTable
 - High performance data storage system built on GFS and other Google technologies
 - Master-slave architecture
 - One key, multiple values
 - Columnar, SSTable (Sorted String Table) Storage, Append-only, Memtable, Compaction
- Amazon DynamoDb
 - Highly available and scalable key-value storage system
 - Decentralized peer to peer architecture
 - Compromise on consistency for better availability - Eventual consistency
 - Consistent hashing, Gossip protocol, Replication, Read repair
- Cassandra
 - Inherited from BigTable and DynamoDb
 - BigTable: Column families, Memtable, SSTable
 - DynamoDb: Consistent hashing, Partitioning, Replication



Introduction

- **Developed by**
 - Avinash Laxman (Co-inventor Amazon DynamoDb)
 - Prashant Malik (Technical Leader at Facebook)
- **Goals:**
 - Distributed NoSQL database (on commodity hardware)
 - Large amount of structured data
 - High availability
 - No single point of failure
- Basic data model is rows & columns
- Column-oriented, Decentralized peer to peer & follow Eventual consistency
- Datastax company develop and support commercial edition of Cassandra



Cassandra Development

- Developed in Java
- 2007-2008 - Developed at Facebook
- July 2008 - Open sourced by Facebook
- March 2009 - Apache Incubator project
- February 2010 - Apache Top-level project
- 2011 - version 0.8 - Added CQL
- 2013 - version 2.0 - Added light-weight transactions, Triggers
- 2015 - version 3.0 - Storage engine improved, Materialized views
- 2017 -version 3.11 – bug fix from the last release
- 2021 - version 4.0.5 ---> we r using this version
- 2022 - version 4.1.0 - Latest release



Installation

- Prerequisite
 - Java 8 (Java 11 experimental)
- Can be installed through apt or yum tool (Ubuntu/CentOS)
- Manual installation
 - Download Cassandra 3.11.x (.tar.gz) and extract it
 - set CASSANDRA_HOME to Cassandra directory
 - set JAVA_HOME to JDK 8 directory
 - Install python 2.7 (for cqlsh)
 - set CASSANDRA_HOME/bin into PATH variable
 - Start Cassandra
- terminal1> cassandra
- terminal2> cqlsh

☾ mapping

mysql	cassandra	
-----	-----	
database	keyspace	
table	table	
row	row	
column	column	



Features

- Peer to peer architecture
- Linear scale performance
- High Performance
- Simplified deployment and maintenance
- Less expensive
- Supports multiple programming languages
- Operational and Development simplicity
- Cloud Availability
- Ability to deploy across data centers
- Fault tolerant
- Configurable consistency (tight or eventual)
- Flexible data model
- Column family store



Limitations

- Aggregation operations are not supported
- Range queries on partition key are not supported
- Not good for too many joins
- Not suitable for transactional data
- During compaction performance / throughput slows down
- Not designed for update-delete



Performance

- Performance measures
 - Throughput (operations per second)
 - Latency
- Cassandra vs MySQL
 - MySQL (more than 50GB data)
 - MySQL (more than 50GB data)
 - Write speed: 300 ms
 - Read speed: 350 ms
 - Cassandra (more than 50GB data)
 - Write speed: 0.12 ms
 - Read speed: 15 ms



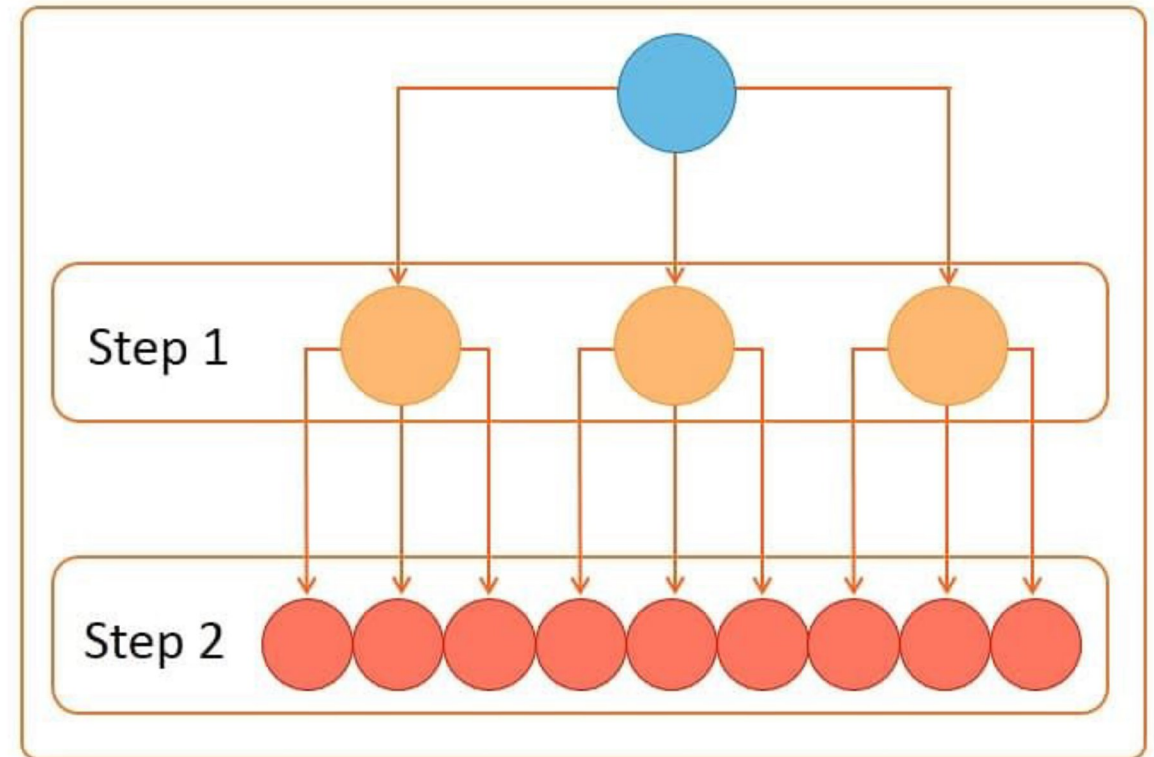
Applications

- Applications
 - Product catalog/Playlist
 - Recommendation/Personalization engine
 - Sensor/IoT data
 - Messaging/Time-series data
 - Fraud detection
- Customers
 - Facebook, Netflix, eBay, Apple, Walmart, GoDaddy
- Application requirements
 - Store and handle time-series data
 - Store and handle large volume of data
 - Scale predictably (Linear Scaling)
 - High availability



Gossip Protocol

- Cassandra uses a gossip protocol to communicate with nodes in a cluster
- It is an inter-node communication mechanism similar to the heartbeat protocol in Hadoop
- Cassandra uses the gossip protocol to discover the location of other nodes in the cluster and get state information of other nodes in the cluster
- The gossip process runs periodically on each node and exchanges state information with three other nodes in the cluster
- Eventually, information is propagated to all cluster nodes. Even if there are 1000s of nodes, information is propagated to all the nodes within a few seconds



Data Model

- Cassandra provides the Cassandra Query Language (**CQL**), an SQL-like language, to create and update database schema and access data
- CQL allows users to organize data within a cluster of Cassandra nodes using:
 - **Keyspace**
 - Defines how a dataset is replicated, per datacenter
 - Replication is the number of copies saved per cluster. Keyspaces contain tables
 - **Table**
 - Defines the typed schema for a collection of partitions. Tables contain partitions, which contain rows, which contain columns. Cassandra tables can flexibly add new columns to tables with zero downtime.
 - **Partition**
 - Defines the mandatory part of the primary key all rows in Cassandra must have to identify the node in a cluster where the row is stored
 - All performant queries supply the partition key in the query.
 - **Row**
 - Contains a collection of columns identified by a unique primary key made up of the partition key and optionally additional clustering keys
 - **Column**
 - A single datum with a type which belongs to a row



- Users can access Cassandra through its nodes using Cassandra Query Language (CQL)
- CQL treats the database (Keyspace) as a container of tables
- Programmers use cqlsh: a prompt to work with CQL or separate application language drivers



Data Types

- ascii: US-ascii character string
- bigint: 64-bit signed long ints
- blob: Arbitrary bytes in hexadecimal
- boolean: True or False
- counter: Distributed counter values 64 bit
- decimal: Variable precision decimal
- double: 64-bit floating point
- float: 32-bit floating point
- frozen: Tuples, collections, UDT containing CQL types
- inet - IP address in ipv4 or ipv6 string format
- int: 32 bit signed integer
- list: Collection 01 elements
- map: JSON style collection of elements
- set: Sorted collection of elements
- text: UTF-8 encoded strings
- timestamp: ID generated with
- date+time: as int/string
- timeuuid: Type 1 uuid
- tuple: A group of 2,3 fields
- uuid: Standard uuid (128-bit)
- varchar: UTF-8 encoded string
- varint: Arbitrary precision integer



Cassandra vs MongoDB: Differences

■ Cassandra

- Dev In java
- Column based
- Cassandra uses a traditional model with a table structure, using rows and columns.
- Cassandra offers an assortment of peers node
- Used in-house query language, CQL
- no internal aggregation framework
- According to the CAP theorem Cassandra is an AP system.

■ MongoDB

- Dev in C++
- Document based
- MongoDB employs an objective-oriented or data-oriented model
- MongoDB uses a single master node.
- queries are structured into JSON fragments
- own aggregation framework
- According to the CAP theorem, MongoDB is a CP system

